

# The Traceable Model

## A Cordwell promotion board, six weeks after the incident

Teams of 3 · 30 to 45 minutes · Covers slides 32 and 35 to 47: registry concepts, registration, aliases and tags, loading by alias, lifecycle, promotion policy and automation, version comparison, best practices, and pitfalls

### The Setup

Six weeks ago, a missed safety escalation turned into a two-day archaeology dig because nobody could prove which run produced the serving model. Cordwell fixed it the boring way: one tracking URI, disciplined logging, and a model registry run on aliases and tags. It worked. The escalation classifier has since been retrained twice, promoted twice, and rolled back once, and every one of those moves left a mark in the registry.

Today your team sits as Cordwell's model promotion board. Three items are on the docket: a new teammate needs the model's history reconstructed from the registry's own event log, an automation PR is waiting for review before it gets merge rights over the aliases, and the on-call runbook for the next 2am incident does not exist yet. Same rule as last time: this is a governance exercise, not a coding exercise. Argue about what the registry should say, and let the record settle who is right.

*Series rule, still in force: blame components and policies, not people. Six weeks ago the missing component was the whole registry. Today the failures are subtler: a pointer that moved when it should not have, a guard that silently checks nothing, and a gate that existed on paper only.*

### Your Team

Same three roles as the first session; assign one per person and rotate at the start of each item so everyone holds each role once.

- **Incident Lead.** Keeps the clock, reads prompts aloud, calls the final answer when time runs short.
- **Scribe.** Writes the board's findings on this handout. The registry keeps its record; the board keeps this one.
- **Challenger.** Must contest at least one board decision per item with a “what breaks if we are wrong” argument. On a promotion board, the Challenger is not a courtesy role; it is the whole point of having a board.

### Item 1 · Registry Forensics (12 minutes)

Slides 32, 36, 38, 41

A new engineer joined the team this morning and asked a reasonable question: “what has this model been through?” Below are the nine events the registry recorded for cordwell-escalation-clf, shuffled. Nothing else is needed: the events themselves force their own order. Reconstruct it.

| Card | Registry event (shuffled: these did NOT happen in this order)                                                                         |
|------|---------------------------------------------------------------------------------------------------------------------------------------|
| D    | @challenger assigned to version 2. Tag set on version 2: validation_status=challenger-ready. Shadow test against live traffic begins. |

| Card | Registry event (shuffled: these did NOT happen in this order)                                                                                             |
|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| A    | Registered model cordwell-escalation-clf created. Version 1 registered from run sweep_r1.                                                                 |
| H    | The challenger's shadow window closes clean. @champion assigned to version 3. Tag set on version 3: validation_status=passed.                             |
| C    | Version 2 registered from run sweep_r2.                                                                                                                   |
| I    | @champion assigned to version 2. Tag set on version 3: rollback_reason=latency_p95 breach.                                                                |
| B    | Tag set on version 1: validation_status=failed (recall below the staffing floor on holdout). At this point version 1 is the only version in the registry. |
| F    | Version 3 registered from run retrain_aug. At this point the registry is serving @champion from version 2.                                                |
| E    | The shadow window closes clean. @champion assigned to version 2. Tag set on version 2: validation_status=passed.                                          |
| G    | @challenger assigned to version 3. Tag set on version 3: validation_status=challenger-ready. A new shadow test begins.                                    |

**1. The true order (write the nine letters):**

---

**2. The registry's final state.** Fill the table exactly as get\_registered\_model and the version tags would report it after the last event:

| Version | Aliases at the end | validation_status | Other tags |
|---------|--------------------|-------------------|------------|
| 1       |                    |                   |            |
| 2       |                    |                   |            |
| 3       |                    |                   |            |

**3. Pointer history:** list, in order, every version @champion has pointed to across this history.

---

**4. The uncomfortable one:** who holds @challenger at the end, and is that the state your board wants? Decide what, if anything, you would change in the registry right now, and write the exact operation.

---



---

**5. The tag trap:** at the end of this history, does version 2's tag record show challenger-ready anywhere? Explain what happened to it, and what that means for how your team should design tag keys.

---



---

## Item 2 · Code Review at Midnight (12 minutes)

Slides 39, 40, 41, 47

A teammate opened this PR at 11:58pm titled “automate challenger promotion, small change, please quick-approve.” It implements the promotion policy from the slides and it is about to get write access to your aliases. Your board does not quick-approve things. The PR contains three real defects and at least one smell. Find the defects, then rank them by how expensive each one is, most expensive first. Every claim needs a slide or a mechanism behind it.

```
from mlflow import MlflowClient

RECALL_FLOOR = 0.80
F1_FLOOR     = 0.42
NAME = "cordwell-escalation-clf"

c = MlflowClient()
exp = c.get_experiment_by_name("cordwell_escalation_triage")
runs = c.search_runs([exp.experiment_id], order_by=["metrics.f1 DESC"])

# Guard: never nominate the sitting champion as its own challenger
champion_versions = {
    v.version
    for v in c.search_model_versions(f"name='{NAME}'")
    if "champion" in v.aliases
}

for run in runs:
    # best F1 first
    recall = run.data.metrics.get("recall", 0)
    f1     = run.data.metrics.get("f1", 0)
    if recall >= RECALL_FLOOR and f1 >= F1_FLOOR:
        versions = c.search_model_versions(f"run_id='{run.info.run_id}'")
        v = max(versions, key=lambda m: int(m.version))
        if v.version in champion_versions:
            continue
        c.set_registered_model_alias(NAME, "challenger", v.version)
        c.set_model_version_tag(NAME, v.version,
                                "validation_status", "challenger-ready")
        print(f"promoted version {v.version} to @challenger")

# Keep the ops dashboard vocabulary in sync
c.transition_model_version_stage(NAME, version=v.version, stage="Staging")
```

**Defect 1, and what actually goes wrong when it runs:**

---

---

**Defect 2, and what actually goes wrong when it runs:**

---

---

**Defect 3, and what actually goes wrong when it runs:**

---

---

**Severity ranking, most expensive first, with one line of reasoning per rank:**

---

---

At least one smell that is not a defect today but that your board would still flag:

---

*Hint for calibration, not for answers: one defect produces the wrong result loudly, one produces the wrong result silently while looking exactly like the safety guard it claims to be, and one works perfectly today while printing its own obituary. Which of those is most expensive is a genuine argument; have it.*

### Item 3 • The 2am Runbook and the Retro (8 minutes)

Slides 37, 39, 42 to 44, 46, 47

Event I in Item 1 happened at 2:07am, and the engineer on call improvised it under pager pressure. Your board's third docket item: make the next rollback boring. Write the standing runbook, at most five steps, that any on-call engineer can execute half-asleep. It must include the exact rollback operation, the tag or tags that preserve the why, one verification step proving serving moved, and the one-line message to the team channel. Assume serving code loads models:/cordwell-escalation-clf@champion and nothing else.

---

---

---

---

**The retro question:** the promotion policy on the slides already lists latency p95 within SLA as a @champion gate. The gate existed on paper. Version 3 was promoted anyway and breached within hours. As a board, name the most likely root cause, and the one logging practice from this module that makes that root cause impossible.

---

---

**The budget vote:** Cordwell's platform team will institutionalize exactly two practices from the best-practices slides this quarter: required tag set enforced in a wrapper, consistent parameter names via a shared PARAMS dict, operational metrics logged beside task metrics, nested runs for sweeps, configs and prompt templates always saved as artifacts, sample predictions with ground truth always saved, or error-analysis plots always logged. Pick your two, and write one sentence each on why yours beat the other five. Ties to real budget meetings are intentional.

---

---

**Debrief (5 minutes, whole room)**

Your Incident Lead reports out, in under 90 seconds:

- Your Item 2 severity ranking, defended in one sentence.
- Your answer to Item 1 question 4: what your board did about @challenger, or why it did nothing.
- Your two budget-vote winners.

## Done When

- The nine events carry a written order and the final-state table is complete for all three versions.
- All three Item 2 defects are identified with mechanisms, ranked, and at least one smell is flagged.
- The runbook has at most five steps and includes the rollback operation, tags, verification, and the channel message.
- The budget vote names exactly two practices with one sentence of defense each.
- The Challenger contested at least one decision per item, and the Scribe recorded the outcome of each contest.